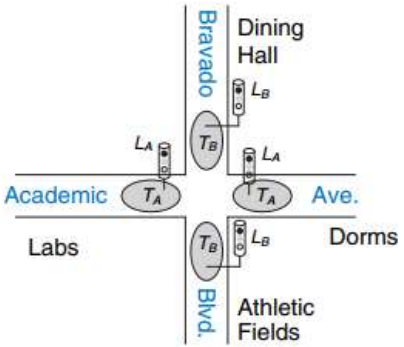
Lab 6: Design sequential circuit (cnt)

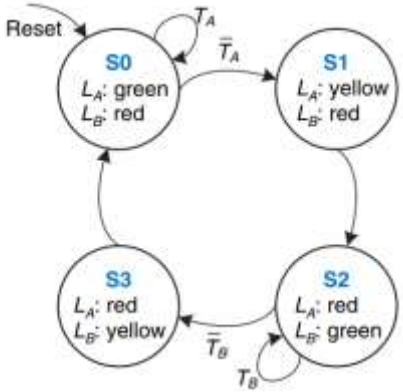
EX1. Traffic light controller

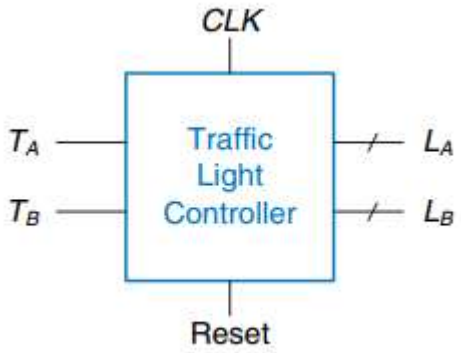
Students move from Labs to Dorms on Academic street.
Football players move from Dining hall to Athletic Fields on Bravado street.

Input: Two traffic sensors, TA and TB. TA indicates true if students are present and false if Academic street is empty. TB indicates true if football player are present and false if Bravado street is empty.

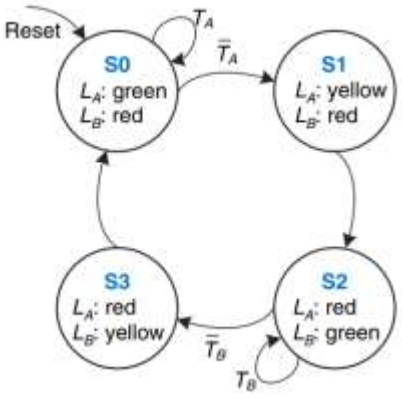
Output: Two traffic lights LA and LB to control traffic. Each lights receives digital inputs to specify whether it should be yellow, red, green.







Design a light traffic controller digital circuit based on following state transition diagram using SystemVerilog and simulate on ModelSim.



Current State S	Inputs T_A	Inputs T_B	Next State S'
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

State transition table

Output	Encoding $L_{1:0}$
green	00
yellow	01
red	10

Output encoding

State	Output
S0	LA: green, LB: red
S1	LA: yellow, LB: red
S2	LA: red, LB: green
S3	LA: red, LB: yellow

Output table

Test D-flipflop on STEP-MAX10 board

Method 2: Using 12MHz clock on STEP-MAX10 board

d_flipflop.sv

```
module d_flipflop(input logic clk, d,
                  output logic q);
 always_ff@(posedge clk)
  begin
        q <= d;
  end



endmodule
```

clock_devider.sv

```
module clock_divider (
    input logic clk_in,    // 12 MHz input clock
    output logic clk_out    // 1 Hz output clock
);

    logic [23:0] counter = 24'd0;  // 24-bit
counter

    always_ff @(posedge clk_in) begin
        if (counter == 24'd6000000) begin
            counter <= 24'd0;
            clk_out <= ~clk_out;    // Toggle
output clock
        end
        else
            counter <= counter + 1;
    end

endmodule
```

top_module.sv

```
module top_module (
    input logic clk_12mhz,  // 12 MHz input
clock
    input logic d ;// D input for D flip-flop
    output logic q  // Q output of D flip-flop
);

    // 1 Hz clock signal
    logic clk_1hz;

    // Instantiate clock divider
    clock_divider clock_divider_instance
(clk_12mhz, clk_1hz);

    // Instantiate D flip-flop
    d_flipflop d_flipflop_instance (clk_1hz,
d, q);

endmodule
```

Ex2:Clock divider

Thiết kế một mạch số bằng SystemVerilog và mô phỏng trên Modelsim dựa trên clock divider để làm cho 1 LED nhấp nháy 2s/ 1 lần

Ex3: Modulo 8 counter

Bảng dưới đây liệt kê một mã Gray 3 bit biểu diễn các số từ 0 đến 7. Hãy thiết kế một máy trạng thái hữu hạn (FSM) đếm mã Gray 3 bit modulo 8 có ba đầu ra:
Thiết kế một mạch số dựa trên sơ đồ và bảng trên bằng SystemVerilog và mô phỏng trên Modelsim.

- Chạy trên mạch max10 hiển thị lên trên LED 7 đoạn.
- Thiết kế them 1 chức năng UP/DOWN để có thể thay đổi được giá trị. Nếu UP = 1, bộ đếm sẽ tiến lên số tiếp theo. Nếu DOWN = 1, bộ đếm sẽ quay lại số trước đó.
- Dựa vào đó hãy thiết kế bộ đếm từ 00 -> 99 (sử dụng bộ chia clock)

Trạng thái hiện tại (State)	Trạng thái kế tiếp (Next State)	Đầu ra (Gray Output) [2:0]	Giá trị thập phân (Tham khảo)
S0	S1	**000**	0
S1	S2	**001**	1
S2	S3	**011**	3
S3	S4	**010**	2
S4	S5	**110**	6
S5	S6	**111**	7
S6	S7	**101**	5
S7	S0	**100**	4

Number	Gray code		
0	0	0	0
1	0	0	1
2	0	1	1
3	0	1	0
4	1	1	0
5	1	1	1
6	1	0	1
7	1	0	0